foofit: X-ray Reflectivity Fitting Software

H.-G. Steinrück[1,*]

[1]IHE-1, Forschungszentrum Jülich & RWTH Aachen

*h.steinrueck@fz-juelich.de

February 2026

Abstract

This document serves as the manual for *foofit*, a pip-installable Python library for X-ray reflectivity (XRR) fitting. *foofit* implements the Parratt recursion formalism [1] and a refraction-corrected master formula, using `lmfit` [2] for optimization and Monte Carlo confidence analysis.

1 Introduction

foofit is a Python library for fitting X-ray reflectivity (XRR) data. It implements two formalisms for computing theoretical XRR curves: the recursive Parratt formalism and a refraction-corrected master formula. Optimization is handled via the `Minimizer` function from `lmfit` [2], and Monte Carlo resampling [3] is used for parameter confidence analysis.

The code was originally developed by Michael Klimczak and Hans-Georg Steinrück at the Crystallography and Structural Physics institute of the Friedrich-Alexander Universität Erlangen-Nürnberg (based on *pyRefFit*), and later refactored into a pip-installable package.

Source code and issue tracker: https://github.com/hgstei/foofit

2 Installation

Install directly from GitHub:

```
pip install git+https://github.com/hgstei/foofit.git
```

All dependencies are installed automatically: `numpy`, `scipy`, `matplotlib`, `lmfit`, `prettytable`, `tqdm`, `corner`, `joblib`, `numdifftools`, `astropy`.

For interactive, zoomable plots in Jupyter, additionally install `ipympl`:

```
pip install ipympl
```

3 Quick Start

```
from foofit import *
```

```
# bundled example dataset: polystyrene on silicon
si_ps_xrr = example_data

# set up lmfit Parameters
params = Parameters()
params.add('numbLayers', value=1,    vary=False)
params.add('wavelength', value=1.0,  vary=False)
params.add('I0',         value=1,    vary=False)
params.add('bkg',        value=0,    vary=False)
params.add('pre_rho',    value=0,    vary=False)
params.add('pre_beta',   value=0,    vary=False)
params.add('layer0_dd',  value=220, min=150, max=300, vary=True)
params.add('layer0_rho', value=0.25, min=0.1, max=0.4, vary=True)
params.add('layer0_sig', value=5,   min=0.5, max=15,  vary=True)
params.add('layer0_beta',value=0,    vary=False)
params.add('sub_rho',    value=0.71, vary=False)
params.add('sub_sig',    value=4,   min=0.5, max=10,  vary=True)
params.add('sub_beta',   value=0,    vary=False)

# single fit
performFit(si_ps_xrr, params, fitFunc=xrr_parratt_fit,
           method='powell', qmin=0.0, qmax=0.4, plot=2,
           outputName='my_sample', weight=2)

# Monte Carlo error analysis
params2 = loadParams('my_sample_<timestamp>_fit.fitParams',
                     lowLim=0.3, highLim=0.3)
performFit_mc(si_ps_xrr, params2, fitFunc=xrr_parratt_fit,
              method='powell', qmin=0.0, qmax=0.4,
              outputName='my_sample', weight=2, NN=250)

# corner plot of MC parameter distributions
analyze_mc('my_sample_<timestamp>_fit_mc.fitParams')
```

A full worked example is provided in `example.ipynb`.

Interactive Plots in Jupyter

Uncomment the appropriate line at the top of the notebook:

```
# %matplotlib widget    # JupyterLab (requires: pip install ipympl)
# %matplotlib notebook  # classic Jupyter Notebook
```

Note: Do not add comments on the same line as a `%matplotlib` magic command — IPython does not treat `#` as a comment in magic arguments. Put the comment on the next line instead.

4 Parameters Convention

All fitting functions take an `lmfit Parameters` object. The parameter names listed below must be followed exactly. Layer index n starts at 0 for the topmost layer (closest to the ambient

medium). Mathematical constraints between parameters can be defined as described in the lmfit documentation [4].

Parameter	Description
`numbLayers`	Number of layers between ambient and substrate (integer, fixed)
`wavelength`	X-ray wavelength in Å (fixed)
`I0`	Intensity scale factor
`bkg`	Background offset
`pre_rho`	Ambient medium electron density (0 for air/vacuum)
`pre_beta`	Ambient medium absorption constant (0 for air/vacuum)
`layer{n}_dd`	Thickness of layer n in Å
`layer{n}_rho`	Electron density of layer n in $e^-/Å^3$
`layer{n}_sig`	Interfacial roughness of layer n in Å
`layer{n}_beta`	Absorption constant of layer n
`sub_rho`	Substrate electron density in $e^-/Å^3$
`sub_sig`	Substrate roughness in Å
`sub_beta`	Substrate absorption constant

5 Functions

`xrr_parratt_calc(params, qq, doConv=0)`

Calculates XRR using the recursive Parratt formalism [1]. Takes an `lmfit Parameters` object, a q-array in $Å^{-1}$, and optionally a Gaussian convolution width `doConv` (sigma in $Å^{-1}$, `0` to disable). Returns the reflectivity array. Cross-checked against GenX [5].

`xrr_master_refractionCorrected_calc(params, qq, doConv=0)`

Same interface as `xrr_parratt_calc`, but uses the refraction-corrected master formula.

`xrr_parratt_fit(params, qq, data, weight, ee, doConv=0)`

Residual function for use with `lmfit`. Returns the weighted difference between model (Parratt) and data. Passed as `fitFunc` to `performFit` and `performFit_mc`.

`xrr_master_refractionCorrected_fit(params, qq, data, weight, ee, doConv=0)`

Same as `xrr_parratt_fit` but using the refraction-corrected master formula.

`xrr_eDens(params, zz)`

Calculates the electron density profile (in $e^-/Å^3$) including interfacial roughness as a function of depth z in Å.

`xrr_eDens_zeroRoughness(params, zz)`

Same as `xrr_eDens` but with sharp (roughness-free) interfaces.

`xrr_beta(params, zz)`

Calculates the absorption (β) profile as a function of depth z in Å.

`xrr_beta_zeroRoughness(params, zz)`

Same as `xrr_beta` but with sharp (roughness-free) interfaces.

`performFit(...)`

```
performFit(dataFile, params, fitFunc=xrr_parratt_fit,
           method='differential_evolution',
           qmin=0, qmax=1, plot=1, report=True, save=True,
           outputName="foo", weight=0, rrfPlot=False, doConv=0)
```

Performs a single fit and optionally plots and saves results.

- `dataFile`: Path to data file. Column 1 must be q in Å^{-1}, column 2 must be intensity I. Additional columns are ignored.
- `fitFunc`: `xrr_parratt_fit` (default) or `xrr_master_refractionCorrected_fit`.
- `method`: Optimization algorithm from `lmfit`. `'powell'` and `'differential_evolution'` work best in practice.
- `qmin`, `qmax`: Fitted q-range in Å^{-1}.
- `plot`: `1` plots XRR and fit only; `2` additionally saves all output files (see below).
- `report`: Print fit report to console.
- `save`: Save report plot to file.
- `outputName`: Base name for all output files.
- `weight`: Figure-of-merit definition. `0`: (data − model)/data; `1`: data − model; `2`: log(data) − log(model); `4`: (data − model)/error.
- `rrfPlot`: If `True`, display Fresnel-normalized reflectivity ($R \cdot q^4$).
- `doConv`: Gaussian convolution width (sigma in Å^{-1}) for instrumental resolution smearing. `0` disables convolution.

Output files saved when `plot=2` (all prefixed `outputName_timestamp_fit`): `.fitParams`, `.ed` (electron density), `.ned` (electron density, no roughness), `.beta`, `.nbeta` (absorption, no roughness), `.r` (reflectivity model curve).

performFit_mc(...)

```
performFit_mc(dataFile, params, fitFunc=xrr_parratt_fit,
              method='differential_evolution',
              qmin=0, qmax=1, plot=1, report=True,
              outputName="foo", weight=0, rrfPlot=False,
              doConv=0, NN=10)
```

Performs Monte Carlo error analysis [3] by generating `NN` noise-perturbed datasets and fitting each independently. Fits are parallelized using `joblib` [6]. Always saves output files and creates a report figure. Similar in approach to MOTOFIT [7].

Parameters are identical to `performFit`, with the addition of:

- `NN`: Number of Monte Carlo iterations (default 10; 100–500 recommended for reliable statistics).

loadParams(parameterFilename, lowLim=0.1, highLim=0.1)

Loads a `.fitParams` file saved by `performFit` back into an `lmfit Parameters` object. `lowLim` and `highLim` define lower and upper parameter bounds as a fraction of the fitted value (e.g., `0.3` gives ±30 % bounds), used as limits in the subsequent Monte Carlo fit.

analyze_mc(file, bins=20)

Generates a corner plot [8] from the `.fitParams` file saved by `performFit_mc`, showing marginal parameter distributions and pairwise correlations.

example_data

Module-level string variable containing the absolute path to the bundled example dataset (`si_ps.xrr`): a polystyrene thin film on a silicon substrate.

6 Physical Constants

The classical electron radius r_e and the critical-q prefactor are computed from `astropy` constants at import time and re-exported by `from foofit import *`:

```
# r_e not directly available in astropy < 4;
# derived from Thomson cross-section: sigma_T = (8*pi/3) * r_e^2
r_e_AA = sqrt(3 * sigma_T / (8*pi)) * 1e10  # = 2.81794e-5  Angstrom
qc_factor = 4 * sqrt(r_e_AA * pi)           # = 0.037636
```

References

[1] L.G. Parratt, Physical Review **95**, 359 (1954).

[2] M. Newville et al., *LMFIT: Non-Linear Least-Square Minimization and Curve-Fitting for Python* (2014), https://lmfit.github.io/lmfit-py/.

[3] F. Heinrich, T. Ng, D.J. Vanderah, P. Shekhar, M. Mihailescu, H. Nanda, and M. Lösche, Langmuir **25**, 4219 (2009).

[4] *lmfit constraints documentation*, https://lmfit.github.io/lmfit-py/constraints.html.

[5] M. Björck and G. Andersson, Journal of Applied Crystallography **40**, 1174 (2007).

[6] *joblib: running Python functions as pipeline jobs*, https://joblib.readthedocs.io/.

[7] A. Nelson, Journal of Applied Crystallography **39**, 273 (2006).

[8] D. Foreman-Mackey, Journal of Open Source Software **1**, 24 (2016).